

Challenge Problem #1

Shellene Redhorse and Reza Parsey

Introduction

The assignment for this challenge problem was to create a program for the management of warehouse robots. Each robot has a randomly assigned beginning and end point, representing where they are starting with their load and where they need to take it to, along with an assigned type of either humanoid, quadrotor, or differential drive. The robots are then expected to travel from their starting position to their end destination in the quickest possible route.

Approach

To approach this problem, we first defined a robot class with subclasses within it. A “Robot” class was created, which contained a randomly assigned initial position, a randomly assigned goal (while ensuring that the goal position was different from the initial position), and a position variable which had its starting value set to the initial position. A return_position call was added to the robot class so that the other classes could easily determine any robot’s position. Subclasses for each of the robot types were also created, which when called would return the name of the robot type so that the other classes could know which type of robot was at a given position.

The “Robot” class also includes a propose_move call which proposes the next step of the robot. It is only a proposed step because it still needs to be checked to see whether that move will satisfy the other conditions. In addition, we have a distance_to_goal call and a reached_goal to return the distance and whether the goal has been reached, respectively. We also included a robot compatibility checker which will return whether two given robot types can share the same location.

In addition, we defined a grid class to run the actual moving of the robots. It takes in values from the main class and assigns these to be the size and width of the grid, initially beginning with an empty one. The next step is to initialize the grid. This is done by randomly selecting a robot type and creating an instance of this type, along with its randomly assigned initial position and goal as created within the robot class, then repeating this process until the number of robots is equal to the number prescribed by the input.

Within the step call is where the robots are actually moved. It first isolates robots which have not yet reached their destination and uses the move proposal call within the Robot class to generate proposed next step destinations for each of them. Using the compatibility checker, we

can now determine whether two given robots will intersect, and the code then ensures that the robot furthest from its goal (using the `distance_to_goal` call) gets to move into that space. Robots which do not get to move are added to a special “blocked set”. There are also considerations for special cases, including robots swapping positions or which have the same goal, and these are allowed to move even if they intersect.

Once this first pass has run, there is an additional check to see if a robot has moved into a space where another robot remained because it did not get to move. This would not have been caught initially because it was only based on where the robot desired to move, not where it actually was able to. With this addition however, we are able to ensure that there are no collisions from this case. The code saves the initial state, then updates the actual positions with the proposed positions once any conflicts have been resolved. This initial state includes details about the type, position, goal, shape, and color of robots which have not yet reached their destinations.

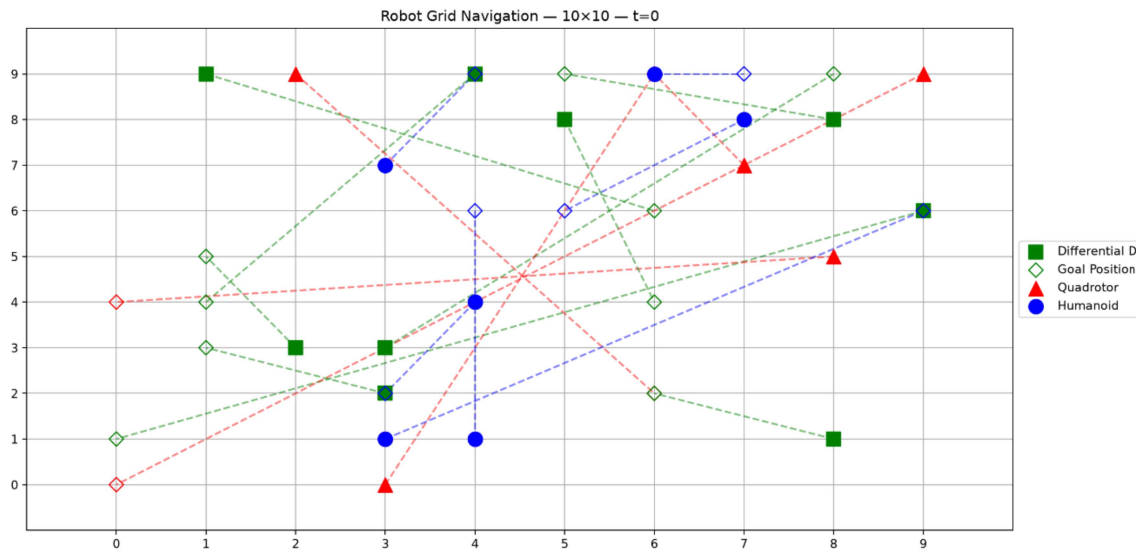
The final step is the visualizer class. It displays the grid with each robot type displayed as a different shape, and different colors representing different iterations. It also displays the robots’ goals as diamonds and lines connecting their current positions to their goal. The arrow keys allow the user to scan through several timesteps, using the states saved within the grid class.

All of these classes are started from the main class. This is where the user can input the grid size and number of robots. It also calls the necessary class functions to create the grid, populate it, run the simulation, and visualize the results.

This code is able to generalize to an $n \times n$ grid via the user inputs, thereby accomplishing the bonus-point goal set out in the problem statement.

Results

The result of our code is that we were able to successfully create a program which takes in the user-defined size of the grid and number of robots, randomly assigns their types, initial positions, and goals, and then calculates their paths and displays them in a visualizer. None of the robots intersect if they are not allowed to, and they all reach their destinations successfully. The visualizer also shows the paths and can step through various time values to show how the robots interact during their journeys. The visualizer is shown below:



Challenges

During certain implementations of the priority-based movement, there were cases where the robots would get into deadlock configurations. Each robot's next required cell was occupied by another who faced similar issues with other robots. This issue particularly arose with the increased grid size. To mitigate this, there was an added deadlock mitigation mechanism that activates when it confirms it is unable to continue. The robot that has been waiting the longest is selected to move to an adjacent cell that is empty (or available to move to) so that the rest of the robots may be able to continue.